

Sistema multiagente con modelos de lenguaje para análisis financiero histórico

Máster en Deep Learning
Proyectos de Deep Learning

Curso 2025–2026

Resumen

Este documento presenta el diseño de un proyecto software que integra modelos de Deep Learning, en concreto modelos de lenguaje de gran escala, dentro de un flujo multiagente para análisis financiero histórico. El objetivo no es construir un asesor financiero ni predecir precios futuros, sino desarrollar un sistema que permita transformar consultas estructuradas sobre activos financieros en análisis reproducibles, código ejecutable y explicaciones comprensibles. El trabajo parte del MVP desarrollado para el TFM: una arquitectura modular que separa ingesta, enrutado, planificación, generación de código, ejecución e interpretación, manteniendo una base determinista y una integración opcional de modelos LLM mediante APIs compatibles con OpenAI.

Índice

1. Introducción	4
1.1. Título del proyecto	4
1.2. Problema que se quiere resolver	4
1.3. Contexto y motivación	4
1.4. Objetivos del proyecto	4
2. Estado del arte	5
2.1. Modelos de lenguaje y uso de herramientas	5
2.2. LLM en el dominio financiero	5
2.3. MLOps y trazabilidad	5
3. Datos	6
3.1. Fuente de los datos	6
3.2. Formato de los datos	6
3.3. Volumen esperado	6
3.4. Necesidad de datos adicionales	6
3.5. Preprocesamiento necesario	7
3.6. Evaluación de la calidad de los datos	7
3.7. Fuentes adicionales	7
4. Enfoque y modelado	8
4.1. Tipos de modelos a evaluar	8
4.2. Justificación del enfoque elegido	8
4.3. Modelos base	8
4.4. Frameworks y librerías	9
5. Entrenamiento	10
5.1. División del dataset	10
5.2. Métricas de evaluación	10
5.3. Optimización	10
5.4. Hardware necesario	10
5.5. Uso de MLOps y seguimiento de experimentos	11
6. Evaluación	12
6.1. Resultados obtenidos en validación y test	12
6.2. Curvas de aprendizaje y sobreajuste	12
6.3. Comparación con métodos base	12
6.4. Pruebas en entorno real	12
6.5. Criterios para lanzar a producción	13
7. Aspectos de ciberseguridad y consideraciones éticas	14
7.1. Posibles ataques al modelo	14
7.2. Medidas de protección	14
7.3. Privacidad de los datos	14
7.4. Sesgos y equidad	14
7.5. Uso responsable	14

8. Despliegue	15
8.1. Plataforma de despliegue	15
8.2. Formato del modelo final	15
8.3. Contenedores y orquestación	15
8.4. Monitorización en producción	15
9. Costes y sostenibilidad	16
9.1. Costes de entrenamiento y despliegue	16
9.2. Uso eficiente de recursos	16
10. Conclusiones	17

1. Introducción

1.1. Título del proyecto

El proyecto se titula *Sistema multiagente con modelos de lenguaje para análisis financiero histórico*. La propuesta se enmarca en el desarrollo de un prototipo software que utiliza agentes especializados, ejecución programática de código y modelos de lenguaje para automatizar análisis descriptivos sobre datos financieros.

1.2. Problema que se quiere resolver

El análisis financiero histórico combina tareas heterogéneas: localizar y cargar datos, validar entradas, seleccionar el tipo de análisis, calcular métricas, generar código, ejecutar dicho código, interpretar resultados y presentar una respuesta útil para el usuario. En muchos sistemas basados en LLM, estas fases se concentran en una única llamada generativa. Ese enfoque puede producir respuestas fluidas, pero dificulta comprobar qué datos se han usado, qué cálculos se han realizado y dónde se ha podido introducir un error.

El problema que se aborda es la falta de trazabilidad y control en prototipos de análisis financiero asistidos por inteligencia artificial. El sistema separa el razonamiento textual de los cálculos verificables, de modo que el modelo de lenguaje pueda aportar valor en la planificación o la explicación sin sustituir las validaciones ni la ejecución controlada.

1.3. Contexto y motivación

Los modelos de lenguaje actuales han demostrado capacidad para razonar sobre instrucciones, generar código y redactar explicaciones adaptadas al usuario. Sin embargo, en dominios sensibles como el financiero resulta peligroso aceptar una respuesta textual sin evidencia intermedia. Incluso cuando el objetivo es descriptivo, una interpretación errónea de rentabilidades, volatilidad o periodos temporales puede inducir decisiones equivocadas.

La motivación del proyecto es construir una arquitectura en la que los LLM se integren dentro de un flujo gobernado por reglas, contratos de datos y ejecución reproducible. La versión inicial del MVP funciona de manera determinista; después se activa el modelo de lenguaje por fases, empezando por la interpretación final. Así se puede medir el impacto del componente generativo sin comprometer la estabilidad del sistema.

1.4. Objetivos del proyecto

El objetivo general es desarrollar y validar un prototipo multiagente capaz de ejecutar análisis financieros históricos a partir de entradas estructuradas y datos en CSV, incorporando modelos de lenguaje de forma gradual y controlada.

Los objetivos específicos son:

- Diseñar un workflow modular que separe ingesta, enrutado, planificación, generación de código, ejecución e interpretación.
- Implementar familias de agentes especializadas para distintas intenciones analíticas.
- Mantener un modo determinista reproducible que sirva como línea base.
- Integrar modelos de lenguaje mediante APIs compatibles con OpenAI, con activación granular y fallback.
- Evaluar el sistema con casos funcionales y escenarios de error controlado.
- Delimitar el alcance ético del sistema para evitar recomendaciones de inversión o predicciones no justificadas.

2. Estado del arte

2.1. Modelos de lenguaje y uso de herramientas

La investigación reciente sobre LLM ha mostrado que estos modelos pueden combinar razonamiento textual con acciones externas. ReAct propone intercalar razonamiento y acciones para que el modelo pueda planificar, consultar herramientas y actualizar su respuesta a partir de observaciones [1]. Toolformer explora cómo un modelo puede aprender a utilizar APIs externas para complementar su generación con resultados procedentes de herramientas [2]. Estas ideas son relevantes para el proyecto porque el análisis financiero exige acceder a datos y ejecutar cálculos, no solo producir texto.

El TFM adopta una visión prudente de este paradigma. El LLM no actúa como una caja negra que decide todo el flujo, sino como un componente dentro de una arquitectura con fases separadas. La planificación, la generación de código y la interpretación pueden apoyarse en modelos de lenguaje, pero los datos, las validaciones y los artefactos de ejecución permanecen observables.

En la misma línea, los sistemas multiagente recientes dividen una tarea compleja en agentes conversacionales o funcionales con responsabilidades acotadas. AutoGen, por ejemplo, propone aplicaciones LLM compuestas por varios agentes que colaboran mediante mensajes y herramientas [3]. Esta idea encaja con el proyecto, aunque aquí se aplica con más control: los agentes no son procesos autónomos sin restricciones, sino módulos especializados con contratos de entrada y salida.

2.2. LLM en el dominio financiero

El uso de modelos de lenguaje en finanzas se ha acelerado con trabajos como FinGPT, que plantea modelos abiertos adaptados al sector financiero [4]. Estos enfoques buscan aprovechar fuentes financieras, noticias, datos de mercado y tareas de análisis específicas del dominio. No obstante, el dominio financiero introduce riesgos particulares: interpretaciones con apariencia de consejo profesional, sesgos en datos históricos, cambios de régimen de mercado y alta sensibilidad a errores numéricos.

El proyecto se diferencia de un sistema de predicción de mercado. Su objetivo es descriptivo y educativo: calcular métricas históricas, comparar activos, estimar retornos y riesgo, y generar explicaciones sobre datos pasados. Esta delimitación reduce el riesgo de uso indebido y permite centrar el trabajo en la arquitectura software. Frente a modelos financieros orientados a recomendación o predicción, el criterio de calidad principal no es anticipar precios, sino mantener fidelidad a los datos, trazabilidad de los cálculos y ausencia de recomendaciones de inversión.

2.3. MLOps y trazabilidad

Las prácticas MLOps proponen registrar experimentos, datos, parámetros, artefactos y métricas para mejorar la reproducibilidad del ciclo de vida de machine learning. Plataformas como MLflow se han popularizado para gestionar ejecuciones, modelos y resultados [5]. En el proyecto, esta filosofía se traslada a un MVP ligero: cada ejecución genera salidas estructuradas, scripts y logs que permiten revisar la relación entre entrada, cálculo y respuesta.

3. Datos

3.1. Fuente de los datos

Los datos utilizados en el MVP proceden de series financieras históricas descargadas y almacenadas localmente en CSV. El MVP desarrollado utiliza ejemplos de activos como NVIDIA, AMD, Apple, Bitcoin, QQQ, SPY, oro, EUR/USD y S&P 500. La fuente prevista para ampliar el sistema es *yfinance*, junto con otros proveedores públicos o APIs financieras cuando sea necesario contrastar datos.

3.2. Formato de los datos

El formato principal es CSV tabular. Cada fichero representa una o varias series temporales con columnas de precio y volumen, fechas o marcas temporales, y metadatos asociados en ficheros JSON. La entrada normalizada del sistema incluye:

- consulta original;
- intención analítica;
- lista de tickers;
- rango temporal o periodo;
- intervalo de muestreo;
- rutas a los CSV que se deben analizar.

3.3. Volumen esperado

La versión actual incluye ocho CSV de datos brutos, con aproximadamente 4.345 filas y un tamaño total cercano a 540 KB. Es un volumen pequeño, adecuado para validar el MVP sin infraestructura pesada. En una versión ampliada, el sistema podría trabajar con varios años de datos diarios o intradía, aumentando a cientos de miles o millones de filas si se incorporan más activos, intervalos horarios y ventanas temporales largas.

Conjunto	Filas aprox.	Tamaño aprox.
Apple tres meses	62	6,5 KB
NVIDIA vs AMD dos años	502	102,6 KB
NVIDIA cinco años	1.257	138,3 KB
Bitcoin 2024	367	33,0 KB
S&P 500 desde 2020	1.560	157,6 KB
EUR/USD diez días a 1h	226	26,9 KB
QQQ y SPY 2024	253	50,5 KB
Oro una semana a 1h	118	11,8 KB

Tabla 1: Volumen de datos brutos disponible en el MVP.

3.4. Necesidad de datos adicionales

Para un prototipo académico, los datos actuales son suficientes para validar el flujo. Para una versión más robusta harían falta:

- mayor cobertura temporal para probar diferentes ciclos de mercado;

- activos de distintos sectores y clases: acciones, índices, divisas, materias primas y criptoactivos;
- datos corporativos o fundamentales si se quisiera ampliar el análisis más allá de precios;
- datos de referencia para comparar resultados con benchmarks externos;
- registros de uso real para evaluar consultas frecuentes y errores de usuario.

3.5. Preprocesamiento necesario

El preprocesamiento incluye normalización de nombres de columnas, conversión de fechas, ordenación temporal, comprobación de valores ausentes, verificación de duplicados, validación de tickers y alineación de series cuando se comparan varios activos. También se requiere controlar cambios de escala, splits, dividendos y diferencias de calendario entre mercados.

3.6. Evaluación de la calidad de los datos

La calidad se evaluará mediante reglas automáticas: presencia de columnas obligatorias, continuidad temporal razonable, porcentaje de valores ausentes, coherencia de precios positivos, detección de outliers extremos y consistencia entre ficheros de datos y metadatos. Las incidencias se guardarán como avisos dentro del estado de ejecución para que el usuario conozca las limitaciones del análisis.

3.7. Fuentes adicionales

Si los datos locales fueran insuficientes se podrían incorporar APIs como Alpha Vantage, Stooq, Nasdaq Data Link o proveedores institucionales. Para mantener reproducibilidad, cada descarga debería registrar fecha de extracción, proveedor, ticker, intervalo, zona horaria y versión del proceso de limpieza.

4. Enfoque y modelado

4.1. Tipos de modelos a evaluar

El proyecto evalúa principalmente modelos de lenguaje de gran escala como componente de Deep Learning. No se plantea entrenar una red neuronal financiera desde cero en la fase inicial, porque el objetivo no es predecir precios sino orquestar análisis reproducibles. Los modelos a evaluar son:

- LLM servidos mediante APIs compatibles con OpenAI;
- modelos abiertos ejecutados en infraestructura universitaria mediante vLLM;
- modelos comerciales o externos como Groq y Gemini para comparar latencia, calidad y estabilidad;
- componentes deterministas como línea base no neuronal.

4.2. Justificación del enfoque elegido

El enfoque multiagente se justifica por la necesidad de separar responsabilidades. Un único prompt podría generar una respuesta, pero sería más difícil auditarlo. La arquitectura propuesta distingue entre:

1. ingesta y validación de datos;
2. enrutado de la intención;
3. selección de agente especializado;
4. generación de plan analítico;
5. generación o selección de código;
6. ejecución controlada;
7. interpretación final.

Esta separación permite activar el LLM solo donde aporte valor. Por ejemplo, en una primera fase se usa para redactar la interpretación a partir de resultados ya calculados. En fases posteriores podría participar en la planificación o en la generación de código, siempre con validaciones y fallback.

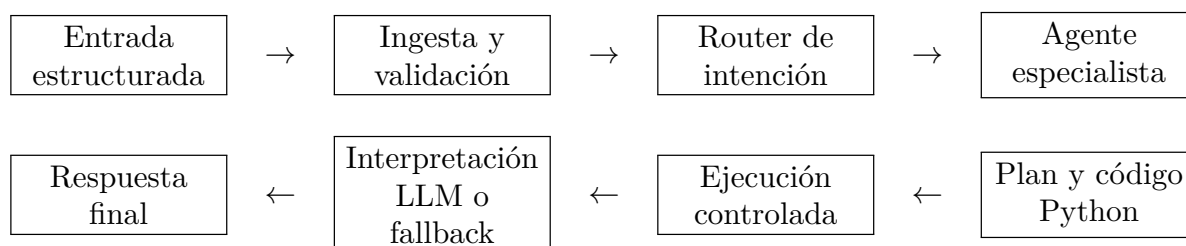


Figura 1: Arquitectura funcional del MVP multiagente.

4.3. Modelos base

Se utilizarán modelos preentrenados mediante transfer learning o APIs ya disponibles. No se propone entrenar desde cero por coste, volumen de datos y falta de necesidad para el objetivo del proyecto. La configuración actual contempla perfiles para Groq, Gemini y un endpoint universitario vLLM con el modelo `openai/gpt-oss-20b`. La capa LLM se activa mediante variables de entorno y puede limitarse a interpretación, planificación o generación de código.

4.4. Frameworks y librerías

El sistema se implementa en Python. Las librerías principales son:

- `pandas` para carga y transformación de datos;
- `numpy` para cálculos numéricos;
- `matplotlib` para gráficos;
- `yfinance` para obtención de datos;
- clientes compatibles con OpenAI para APIs LLM;
- `unittest` o `pytest` para pruebas automatizadas;
- MLflow o Weights & Biases como extensión MLOps futura.

5. Entrenamiento

5.1. División del dataset

Dado que el sistema no entrena inicialmente un modelo predictivo sobre series temporales, no se aplica una división clásica de entrenamiento, validación y test para aprender parámetros. En su lugar se define una separación por casos de evaluación:

- conjunto de desarrollo: consultas usadas para construir y depurar el flujo;
- conjunto de validación: consultas representativas de las seis intenciones soportadas;
- conjunto de test: consultas nuevas y escenarios de error para comprobar robustez.

Si en una fase posterior se entrenara o ajustara un clasificador de intenciones, sí se usaría una división estratificada por tipo de consulta. Para series temporales, cualquier división predictiva debería respetar el orden temporal y evitar fugas de información.

En este trabajo, el apartado de entrenamiento se entiende como preparación y validación del sistema de Deep Learning integrado, no como ajuste de pesos. La unidad experimental es una consulta completa: entrada estructurada, agente seleccionado, código generado, ejecución y respuesta final. Esta decisión mantiene el trabajo alineado con el objetivo de la asignatura, porque el componente neuronal existe como LLM preentrenado integrado en un proyecto software real.

5.2. Métricas de evaluación

Las métricas se adaptan al tipo de componente:

- **Workflow:** tasa de ejecución correcta, errores controlados y cobertura de intenciones.
- **Cálculo:** coincidencia con resultados esperados, ausencia de excepciones y coherencia de unidades.
- **LLM:** utilidad de respuesta, fidelidad a los datos, ausencia de recomendaciones financieras y tasa de fallback.
- **Sistema:** latencia, coste por ejecución, trazabilidad de artefactos y estabilidad ante entradas inválidas.

5.3. Optimización

En la fase actual no se optimizan pesos neuronales. La optimización se centra en el diseño del pipeline: reducir errores de enrutado, mejorar contratos de salida, limitar llamadas innecesarias al LLM y ajustar prompts para que el modelo respete los resultados calculados. Si se ajustara un modelo pequeño para clasificación de intenciones, se emplearían técnicas estándar como regularización, early stopping y búsqueda de hiperparámetros.

5.4. Hardware necesario

El MVP determinista se ejecuta en CPU. La integración LLM mediante APIs externas no requiere GPU local. Si se usa el endpoint universitario con vLLM, el coste de GPU se desplaza al servidor. Para servir un modelo abierto de forma local o privada sería recomendable una GPU con memoria suficiente, dependiendo del tamaño del modelo y de la cuantización.

5.5. Uso de MLOps y seguimiento de experimentos

El seguimiento de experimentos registrará proveedor LLM, modelo, parámetros de activación, caso evaluado, latencia, estado, avisos, uso de fallback y respuesta final. En una versión ampliada, MLflow permitiría centralizar estos artefactos y comparar perfiles deterministas, Groq, Gemini y vLLM universitario.

6. Evaluación

6.1. Resultados obtenidos en validación y test

La versión actual del MVP soporta seis intenciones analíticas: crecimiento de precio, comparación de activos, visión descriptiva general, análisis de retornos, riesgo histórico y análisis técnico. La batería principal de pruebas incluye ejecuciones correctas y escenarios negativos. En el estado actual del repositorio se documenta una validación de 9 pruebas superadas sobre 9.

Tipo	Cobertura	Criterio comprobado	Estado
Funcional	6 intenciones	estado completado y código 0	6/6
Errores	intención no soportada	mensaje controlado	1/1
Errores	CSV inexistente	mensaje controlado	1/1
Errores	cardinalidad técnica inválida	mensaje controlado	1/1

Tabla 2: Resumen de pruebas automatizadas del MVP.

La validación funcional cubre los casos `price_growth`, `compare_assets`, `asset_overview`, `return_analysis`, `historical_risk_analysis` y `technical_analysis`. En todos ellos se comprueba que el workflow termina en estado `completed`, que el script generado devuelve código de salida 0 y que la respuesta contiene información esperada del activo o del tipo de análisis.

También se ha probado la integración del perfil universitario con activación del LLM solo para interpretación. En los seis casos representativos el flujo finalizó correctamente, sin fallback, y sin generar recomendaciones directas de inversión. La latencia media observada fue de aproximadamente 6,76 segundos por consulta, con un rango entre 4,71 y 9,87 segundos.

Perfil	Casos	Completados	Fallbacks	Latencia media
Determinista	6	6	0	baja, local
Universidad vLLM	6	6	0	6,76 s

Tabla 3: Comparación resumida entre baseline determinista e interpretación con LLM.

6.2. Curvas de aprendizaje y sobreajuste

No hay curvas de aprendizaje en el sentido clásico porque no se entrena un modelo predictivo. Sin embargo, sí puede analizarse el sobreajuste de prompts o reglas: un pipeline podría funcionar bien con los ejemplos de desarrollo y fallar con consultas nuevas. Para evitarlo, se propone ampliar el conjunto de test con variaciones de lenguaje, tickers, periodos, intervalos y errores deliberados.

6.3. Comparación con métodos base

La comparación principal se realiza contra el modo determinista. Este modo actúa como línea base porque ofrece respuestas reproducibles y controladas. El LLM se considera superior solo si mejora la claridad, cobertura o adaptabilidad sin introducir errores, recomendaciones financieras ni contradicciones con los resultados numéricos. En la validación actual, el LLM no cambia los cálculos: recibe los resultados estructurados ya ejecutados y redacta una explicación final, lo que reduce el riesgo de alucinación numérica.

6.4. Pruebas en entorno real

Para probar el sistema en un entorno real se debería desplegar una API interna y registrar consultas reales de usuarios académicos o analistas. Las pruebas incluirían validación de entradas,

tiempo de respuesta, tasa de errores, calidad percibida de las explicaciones y revisión humana de casos con potencial ambigüedad. También se deben simular fallos de proveedor LLM, CSV corruptos, latencias elevadas y datos incompletos.

6.5. Criterios para lanzar a producción

El rendimiento sería válido para un entorno real si se cumplen estas condiciones:

- cobertura suficiente de intenciones frecuentes;
- tasa baja de errores no controlados;
- trazabilidad completa de datos, plan, código y resultado;
- ausencia de recomendaciones de inversión;
- latencia aceptable para el usuario;
- fallback fiable cuando falle el LLM;
- revisión de seguridad para la ejecución de código generado.

Las principales limitaciones actuales son el tamaño reducido de la batería de evaluación, la ausencia de pruebas con usuarios reales, la dependencia de proveedores LLM externos cuando se activa la interpretación generativa y la necesidad de reforzar el entorno de ejecución antes de permitir generación libre de código. Estas limitaciones no invalidan el MVP, pero sí marcan qué faltaría antes de considerarlo un sistema productivo.

7. Aspectos de ciberseguridad y consideraciones éticas

7.1. Posibles ataques al modelo

El sistema puede verse afectado por ataques propios de aplicaciones con LLM:

- **Prompt injection:** el usuario intenta forzar al modelo a ignorar instrucciones o producir asesoramiento financiero.
- **Evasion adversarial:** entradas diseñadas para saltarse validaciones o activar un agente incorrecto.
- **Data poisoning:** incorporación de CSV manipulados para sesgar los resultados.
- **Model stealing:** abuso de la API para inferir comportamiento del modelo o prompts internos.
- **Code injection:** intento de introducir rutas, comandos o fragmentos que acaben en el código generado.

7.2. Medidas de protección

Las medidas previstas son:

- validación estricta del esquema de entrada;
- listas de intenciones soportadas y restricciones por agente;
- ejecución de código en proceso controlado y con rutas acotadas;
- separación entre resultados calculados y redacción del LLM;
- limitación de tokens, timeouts y reintentos;
- logging de errores y avisos;
- fallback determinista ante respuestas no válidas del modelo.

7.3. Privacidad de los datos

Los datos financieros de mercado no son datos personales. Aun así, si el sistema evolucionara para aceptar carteras, preferencias o historiales de usuario, sería necesario aplicar minimización de datos, anonimizar identificadores y evitar enviar información sensible a proveedores externos sin consentimiento.

7.4. Sesgos y equidad

Los datos históricos pueden reflejar sesgos de mercado, periodos anómalos o activos con liquidez desigual. El sistema debe explicar que los resultados dependen del periodo analizado y no extrapolar comportamientos pasados como garantía futura. También debe evitar lenguaje categórico que pueda interpretarse como recomendación personalizada.

7.5. Uso responsable

El proyecto se limita al análisis descriptivo. No genera instrucciones de compra o venta, no estima rentabilidad futura y no sustituye asesoramiento financiero profesional. Esta frontera debe aparecer tanto en la documentación como en las respuestas generadas.

8. Despliegue

8.1. Plataforma de despliegue

La opción recomendada es un despliegue cloud ligero o un servidor universitario. El backend se expondría mediante una API REST o una interfaz de línea de comandos evolucionada. Para uso docente, bastaría un servidor local con CPU y acceso configurado a las APIs LLM. Para uso más intensivo, se podría desplegar en AWS, GCP o Azure.

8.2. Formato del modelo final

No existe un único fichero de modelo entrenado. El “modelo final” del sistema es la combinación de código, configuración, agentes, prompts y proveedor LLM seleccionado. Si se incorporara un modelo abierto local, se serviría mediante vLLM y pesos en formato compatible con Hugging Face.

8.3. Contenedores y orquestación

El despliegue se puede empaquetar con Docker, incluyendo dependencias Python, estructura de carpetas, variables de entorno y comandos de ejecución. Kubernetes solo sería necesario si se requiere escalado, alta disponibilidad o separación entre servicios de API, ejecución y monitorización.

8.4. Monitorización en producción

La monitorización debe incluir:

- número de consultas por intención;
- latencia por fase;
- tasa de errores y fallbacks;
- coste aproximado de llamadas LLM;
- proveedor y modelo utilizados;
- avisos de calidad de datos;
- trazas de ejecución para auditoría.

9. Costes y sostenibilidad

9.1. Costes de entrenamiento y despliegue

El coste directo para el desarrollo realizado hasta ahora ha sido nulo. Una de las decisiones del MVP ha sido comprobar hasta dónde se puede llegar sin gastar dinero: modo determinista en CPU local, datos descargados de fuentes gratuitas, librerías abiertas y uso opcional de APIs solo cuando existe acceso disponible. Además, no se ha entrenado un modelo desde cero ni se han ajustado pesos, así que no hay coste propio de entrenamiento.

Los costes potenciales aparecerían al escalar el sistema o al usar de forma continuada modelos grandes. En particular, la universidad ha habilitado un nodo donde se aloja un modelo de mayor tamaño mediante vLLM. Para este trabajo ese recurso no supone un coste directo para el estudiante, pero sí tiene un coste real de infraestructura que convendría solicitar a la universidad antes de la memoria final: tipo de GPU, horas aproximadas de uso, consumo energético, coste de amortización o tarifa equivalente en cloud. Esa estimación permitiría completar el análisis económico con mayor precisión.

Los costes principales a considerar en una versión ampliada son:

- llamadas a APIs LLM externas;
- uso de GPU si se sirve un modelo abierto con vLLM;
- almacenamiento de logs y artefactos;
- mantenimiento del entorno de ejecución.

Para un uso académico con pocos casos, el modo determinista y las llamadas ocasionales a API son suficientes. Para un piloto con usuarios reales, conviene estimar coste por consulta y aplicar límites de uso.

9.2. Uso eficiente de recursos

La sostenibilidad se mejora evitando llamadas LLM innecesarias. El sistema puede ejecutar cálculos en CPU y reservar el modelo de lenguaje para fases donde aporte valor real. También se puede cachear resultados, reutilizar datos descargados, limitar el tamaño de contexto enviado al LLM y registrar qué tareas se resuelven correctamente sin componente generativo.

10. Conclusiones

La idea principal del TFM es construir un sistema multiagente analista que funcione correctamente de extremo a extremo. La contribución central no es predecir mercados en esta fase, sino diseñar e implementar una arquitectura capaz de recibir una consulta estructurada, seleccionar el agente adecuado, cargar datos financieros, generar un plan de análisis, ejecutar cálculos verificables y producir una explicación final. En este sentido, el proyecto integra modelos de Deep Learning dentro de una arquitectura software realista, manteniendo una posición prudente ante el dominio financiero.

El MVP ya cuenta con una base funcional: seis intenciones analíticas, pruebas automatizadas, gestión de errores, generación y ejecución de scripts, y una capa LLM configurable. Los siguientes pasos naturales son ampliar el conjunto de evaluación, reforzar la seguridad de la ejecución de código, registrar experimentos con una herramienta MLOps y comparar de forma sistemática el modo determinista frente a distintos proveedores LLM.

Como trabajo futuro inmediato, el sistema debería incorporar una capa de procesamiento de lenguaje natural que permita recibir peticiones en texto libre. Esa capa tendría que extraer intención, tickers, periodos, intervalos y restricciones de la consulta del usuario, para convertirlas en la entrada estructurada que el MVP ya sabe ejecutar. También se plantea automatizar la obtención de datos mediante *yfinance*, de modo que el sistema pueda descargar por sí mismo las series necesarias cuando no existan en local, registrando proveedor, fecha de descarga y metadatos para conservar la reproducibilidad.

A más largo plazo, una vez consolidado el sistema analista y su trazabilidad, se podría estudiar la incorporación de modelos de *forecasting*. Esta extensión debería tratarse como una fase diferenciada, con división temporal de datos, métricas predictivas, comparación con baselines clásicos y comunicación clara de incertidumbre, evitando que el sistema descriptivo actual se confunda con asesoramiento financiero o predicción garantizada.

Referencias

- [1] Yao, S. et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629, 2022. <https://arxiv.org/abs/2210.03629>
- [2] Schick, T. et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. arXiv:2302.04761, 2023. <https://arxiv.org/abs/2302.04761>
- [3] Wu, Q. et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv:2308.08155, 2023. <https://arxiv.org/abs/2308.08155>
- [4] Yang, H. et al. *FinGPT: Open-Source Financial Large Language Models*. arXiv:2306.06031, 2023. <https://arxiv.org/abs/2306.06031>
- [5] Zaharia, M. et al. *Accelerating the Machine Learning Lifecycle with MLflow*. IEEE Data Engineering Bulletin, 2018. <https://mlflow.org/>
- [6] Aroussi, R. *yfinance Python Library*. GitHub repository. <https://github.com/ranaroussi/yfinance>